

---

# **Rapport Projet de fin d'année : Chatbot SeaTech**

**Réalisé par :**

**Elazzouzi Asmae**

**Ben Romdhane FARAH**

**Encadré par :**

**M.Dinar Farid**

**Année universitaire :**

**2024/2025**

## Table des matières

|                                                        |          |
|--------------------------------------------------------|----------|
| <b>Introduction.....</b>                               | <b>1</b> |
| <b>1. Contexte.....</b>                                | <b>1</b> |
| <b>2. Objectifs.....</b>                               | <b>1</b> |
| <b>3. Problématique et Besoins.....</b>                | <b>1</b> |
| 3.1 Les défis du site existant.....                    | 1        |
| 3.2 Pourquoi un chatbot ?.....                         | 2        |
| <b>4. Développement du Chatbot.....</b>                | <b>2</b> |
| 4.1 Choix de la Méthodologie : RAG vs Fine-Tuning..... | 2        |
| a) Fine-Tuning.....                                    | 2        |
| b) Retrieval-Augmented Generation (RAG).....           | 2        |
| c) Pourquoi avons-nous choisi RAG ?.....               | 2        |
| 4.2 Extraction et Préparation des Données.....         | 3        |
| a) Méthodes utilisées.....                             | 3        |
| b) Nettoyage des données.....                          | 3        |
| 4.3. Chunking.....                                     | 3        |
| 4.4. Transformation en Embeddings.....                 | 3        |
| 4.5. Recherche et récupération d'informations.....     | 4        |
| 4.6. Génération de réponses avec LLaMA 3.....          | 4        |
| <b>5. Tests et choix des outils.....</b>               | <b>4</b> |
| 5.1. Problèmes rencontrés.....                         | 4        |
| 5.2. Solution adoptée : Groq.....                      | 5        |
| <b>6. Déploiement et gestion du projet.....</b>        | <b>6</b> |
| 6.1. Interface utilisateur avec Streamlit.....         | 6        |
| 6.2. Déploiement via Streamlit Cloud.....              | 6        |
| 6.3. Gestion du code avec GitHub.....                  | 7        |
| <b>Conclusion.....</b>                                 | <b>8</b> |

# **Introduction**

L'école d'ingénieurs Seatech dispose d'un site internet destiné à informer les étudiants, enseignants et visiteurs sur divers aspects académiques et administratifs. Cependant, malgré la richesse des informations disponibles, la navigation sur le site peut s'avérer fastidieuse, en particulier pour les nouveaux utilisateurs qui recherchent des réponses précises et rapides. Pour pallier ces limitations, nous avons entrepris le développement d'un chatbot interactif capable d'assister les utilisateurs en leur fournissant des réponses instantanées et adaptées à leurs besoins. Ce chatbot vise à améliorer l'expérience utilisateur en facilitant l'accès aux informations importantes sans nécessiter une recherche approfondie sur le site.

## **1. Contexte**

L'accès aux informations sur le site de Seatech repose sur une navigation classique via menus et pages statiques. Cependant, cette approche peut poser des difficultés aux utilisateurs qui souhaitent obtenir des réponses rapides sans parcourir plusieurs pages. Le chatbot vient répondre à ce besoin en proposant une interaction plus naturelle et intuitive.

## **2. Objectifs**

Le projet a plusieurs objectifs. Il vise tout d'abord à fournir un accès rapide et efficace aux informations du site Seatech, permettant ainsi aux utilisateurs d'obtenir des réponses sans avoir à effectuer des recherches longues et complexes. En automatisant la réponse aux questions fréquentes, il réduit la charge de travail de l'administration qui reçoit souvent des demandes redondantes. Enfin, le projet a également une visée expérimentale, nous permettant de comparer différentes technologies pour la mise en place d'un chatbot performant et adapté aux besoins des utilisateurs.

## **3. Problématique et Besoins**

### **3.1 Les défis du site existant**

Bien que le site internet contient une grande quantité d'informations, plusieurs défis se posent. Tout d'abord, la navigation peut être complexe, avec des menus et sous-menus imbriqués qui rendent difficile l'accès rapide à l'information. Ensuite, la recherche d'informations peut être chronophage, nécessitant plusieurs clics et parfois même une exploration approfondie des pages. Enfin, le besoin d'un support interactif se fait de plus en plus ressentir, notamment pour répondre aux questions récurrentes des étudiants et visiteurs en temps réel.

## **3.2 Pourquoi un chatbot ?**

Le développement d'un chatbot répond à ces problématiques en proposant une interface conversationnelle intuitive. Il permet une interaction plus naturelle et fluide avec les utilisateurs, leur offrant ainsi une expérience plus agréable et efficace. Grâce à cette technologie, la recherche d'informations devient simplifiée et instantanée, évitant ainsi aux utilisateurs de parcourir de multiples pages pour obtenir une réponse. De plus, l'automatisation du support utilisateur permet de soulager l'administration en réduisant considérablement le nombre de demandes répétitives.

## **4. Développement du Chatbot**

### **4.1 Choix de la Méthodologie : RAG vs Fine-Tuning**

#### **a) Fine-Tuning**

Le fine-tuning consiste à entraîner un modèle de langage sur un ensemble de données spécifiques afin d'améliorer ses performances sur un domaine particulier. Cette approche offre des résultats précis mais présente plusieurs inconvénients. Tout d'abord, elle nécessite des ressources importantes pour entraîner et mettre à jour le modèle, ce qui peut engendrer un coût computationnel élevé. De plus, une fois entraîné, le modèle devient rigide et ne peut plus s'adapter facilement aux mises à jour des données sans un nouvel entraînement. Enfin, le stockage et la gestion du modèle peuvent devenir complexes, surtout lorsque les données évoluent fréquemment.

#### **b) Retrieval-Augmented Generation (RAG)**

La méthode RAG repose sur la combinaison d'une recherche d'informations dans une base de données et d'un modèle génératif. Ce processus se déroule en deux étapes. La première est la recherche d'informations, où le modèle récupère les passages les plus pertinents depuis une base de documents. La seconde est la génération de texte, où le modèle utilise ces passages pour produire une réponse cohérente et pertinente, garantissant ainsi une meilleure précision dans les réponses fournies.

#### **c) Pourquoi avons-nous choisi RAG ?**

Nous avons opté pour la méthode RAG car elle permet d'exploiter directement les informations du site sans nécessiter de réentraînement du modèle, ce qui représente un gain de temps considérable. Elle garantit également des réponses plus dynamiques et toujours à jour, car le modèle puise les informations dans une base actualisée. Enfin, cette approche réduit les coûts computationnels tout en assurant une meilleure précision des réponses, en évitant les erreurs liées à des informations obsolètes.

## 4.2 Extraction et Préparation des Données

### a) Méthodes utilisées

L'extraction des données a été une étape cruciale pour alimenter notre base documentaire. Nous avons testé plusieurs méthodes avant d'opter pour la solution la plus efficace. Tout d'abord, nous avons utilisé BeautifulSoup, une bibliothèque Python permettant d'extraire et de parser le contenu HTML des pages web. Cependant, nous avons constaté que la structuration des données obtenues était limitée et nécessitait un post-traitement fastidieux. Ensuite, nous avons expérimenté avec Firecrawl, un outil avancé qui nous a permis d'automatiser l'extraction des données en générant un fichier JSON bien organisé contenant toutes les informations nécessaires issues du site.

### b) Nettoyage des données

Après l'extraction, nous avons entrepris un nettoyage manuel des données afin d'assurer leur qualité et leur pertinence. Cette étape consistait à supprimer les informations redondantes ou inutiles, vérifier la cohérence des textes extraits et organiser les données pour une intégration optimale dans notre pipeline RAG. Un bon nettoyage des données est indispensable pour garantir des résultats précis et fiables lors de l'interrogation du chatbot.

## 4.3. Chunking

Le chunking est une étape cruciale qui consiste à diviser les données textuelles en segments plus petits et cohérents, facilitant ainsi leur indexation et récupération.

### Méthode utilisée :

- Chargement des données à partir d'un fichier **JSON**.
- Extraction des sections **pertinentes** pour conserver uniquement le contenu utile.
- Découpage en **blocs cohérents** pour améliorer la précision des réponses générées.

## 4.4. Transformation en Embeddings

Une fois les segments créés, ils sont transformés en représentations numériques appelées embeddings pour permettre une recherche efficace.

### Méthode utilisée :

- Conversion des chunks en **vecteurs de dimension réduite**.
- Utilisation de **ChromaDB**, une base de données spécialisée dans le stockage et la recherche d'embeddings.
- Attribution d'identifiants uniques à chaque vecteur pour un accès rapide lors des requêtes utilisateurs.

## 4.5. Recherche et récupération d'informations

Une fois les embeddings stockés, le chatbot doit retrouver les informations les plus pertinentes en fonction de la requête de l'utilisateur.

- **Définition d'un LLM (Large Language Model)**

Un **LLM** est un modèle d'intelligence artificielle conçu pour comprendre et générer du texte en langage naturel. Il s'appuie sur de vastes bases de données textuelles pour produire des réponses cohérentes et précises.

- **Processus de récupération d'informations :**

1. L'utilisateur saisit une question dans l'interface.
2. **ChromaDB** recherche les embeddings les plus proches de la requête.
3. Les **3 documents les plus pertinents** sont extraits.
4. Ces documents sont ensuite transmis au **LLM** pour générer une réponse contextuelle.

## 4.6. Génération de réponses avec LLaMA 3

Une fois les documents récupérés, le **modèle LLaMA 3** est utilisé pour générer une réponse adaptée à la requête de l'utilisateur.

- **Méthode utilisée :**

- Prise en compte de la question de l'utilisateur.
- Intégration des informations extraites depuis **ChromaDB**.
- Génération d'une réponse fluide et cohérente en s'appuyant sur les documents récupérés.

👉 **Avantages de LLaMA 3 :**

- ✓ Réponses plus précises et contextuelles.
- ✓ Modèle optimisé pour un traitement rapide et efficace.
- ✓ Adaptabilité aux nouvelles informations sans réentraînement.

## 5. Tests et choix des outils

### 5.1. Problèmes rencontrés

Durant la phase de développement, plusieurs défis ont émergé concernant l'intégration des différents composants et l'optimisation des performances. Parmi les principaux problèmes rencontrés, nous pouvons citer la gestion de la compatibilité entre les outils, la complexité des tests sur des environnements variables, et la difficulté d'assurer une cohérence entre la solution développée et les attentes initiales. De plus, le choix de la plateforme pour le

déploiement a posé des questions sur l'évolutivité du projet et la manière de garantir une expérience utilisateur fluide malgré des ressources limitées.

```
62 # Créer le modèle
63 model = OllamaLLM(model="deepseek-r1:7t")
64
65 # créer la chaîne avec le prompt et le modèle
66 chain = prompt | model
67
68 # Réorganiser les documents obtenus sous forme de texte
69 retrieved_docs = "\n".join(results['documents'][0])
70
71 # Générer la réponse en fonction de la question et des documents trouvés
72 response = chain.invoke({"question": query_text, "documents": retrieved_docs})
```

PROBLEMS OUTPUT DEBUG CONSOLE **TERMINAL** PORTS

```
PS C:\Users\nouno\OneDrive\Bureau\Asmae> & C:/Users/nouno/AppData/Local/Microsoft/WindowsApps/python3.11.exe c:/Users/nouno/OneDrive/Bureau/Asmae/rag
.py
10 chunks ajoutés à ChromaDB.
Résultats de la requête:
1. Admission sur Concours

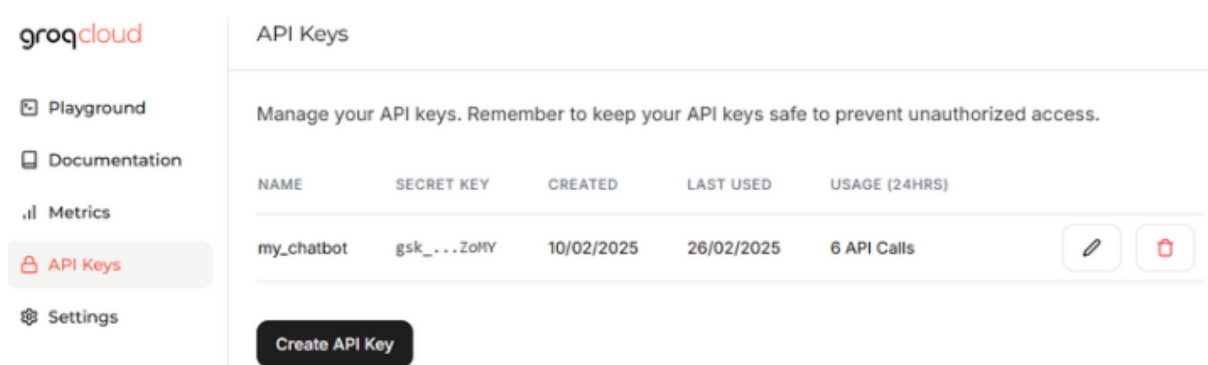
Pour devenir ingénieur SeaTech (hors alternance) **91 places** sont réservées aux concours pour les élèves issus de classes préparatoires.

### Admission sur concours :

**91 places aux Concours Communs INP (CCINP)** pour les élèves issus de classes préparatoires réparties comme suit :
- 30 places Physique et Sciences de l'Ingénieur (PSI)
- 23 places Mathématiques et Physique (MP)
- 23 places Physique et Chimie (PC/PH)
- 9 places Physique et Technologie (PT)
- 2 places Technologie Physique et Chimie (TPC)
- 2 places Technologie et Sciences Industrielles (TSI)
- 2 places Mathématiques, Physique et Informatique (MPI)
```

## 5.2. Solution adoptée : Groq

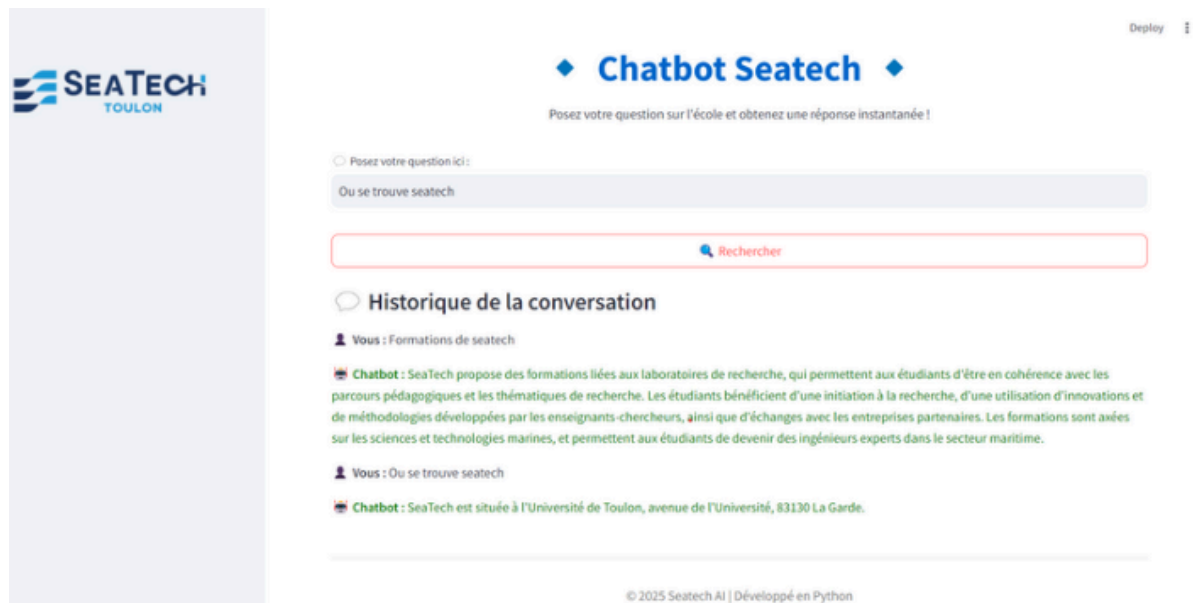
Pour surmonter ces difficultés, nous avons opté pour **Groq** comme solution technique pour le calcul haute performance. Ce choix a été motivé par la capacité de Groq à offrir des performances accrues en matière de traitement de données massives, ce qui était essentiel pour répondre aux besoins de notre projet. Groq permet une exécution parallèle optimisée, réduisant ainsi considérablement le temps nécessaire pour le traitement des données tout en assurant une précision et une fiabilité accrues.



## 6. Déploiement et gestion du projet

### 6.1. Interface utilisateur avec Streamlit

L'interface utilisateur a été développée avec **Streamlit**, un framework Python qui permet de créer rapidement des applications web interactives. Streamlit a été choisi en raison de sa simplicité d'utilisation, de sa compatibilité avec les bibliothèques Python existantes et de sa capacité à générer une interface épurée et réactive. Grâce à Streamlit, l'application est non seulement fonctionnelle mais aussi conviviale, facilitant ainsi l'interaction avec l'utilisateur final.



### 6.2. Déploiement via Streamlit Cloud

Le déploiement de l'application a été effectué sur **Streamlit Cloud**, une plateforme qui permet de déployer facilement des applications Streamlit dans le cloud. Cette solution a été privilégiée en raison de sa simplicité de configuration et de gestion des déploiements, tout en garantissant une haute disponibilité et une scalabilité des ressources. Grâce à Streamlit Cloud, l'application est accessible de manière instantanée par les utilisateurs finaux, et les mises à jour sont déployées de façon transparente.



chatbot\_sesitech - main - chatbot.py

chatbot\_seatedch\_soutenance · main · chatbot.py

### Get started from a template



GDP dashboard

[View demo](#)

Chatbot

[View demo](#)

### Existing tickets

Number of tickets: 100

 Insert all the tickets by double clicking on a cell. When the status below column automatically changes you can stop the sale by clicking on the column headers.

| ID         | Ty               | Name                                         | Ty               | Status | Is priority | Date (calendar) |
|------------|------------------|----------------------------------------------|------------------|--------|-------------|-----------------|
| 1000010001 | Relax            | Relax performance with orchestra             | Relax            | used   | 0           | 2020-07-07      |
| 1000010002 | Live performance | Live performance and relaxing vocalizations  | Live performance | used   | 0           | 2020-07-08      |
| 1000010003 | Music            | Music: guitar - playing improvisation skills | Music            | used   | 0           | 2020-07-10      |
| 1000010004 | Instrument       | Instrument: electronic - bass                | Instrument       | used   | 0           | 2020-07-11      |
| 1000010005 | Music            | Music performance: amplified                 | Music            | used   | 0           | 2020-07-12      |
| 1000010006 | Music            | Music performance: improvisation             | Music            | used   | 0           | 2020-07-12      |

### Support tickets

[View demo](#)

La gestion du code a été assurée via **GitHub**, un outil incontournable pour le suivi des versions et la collaboration en équipe. GitHub a permis de centraliser le code source, de gérer les différentes branches de développement, et de collaborer efficacement grâce aux **pull requests** et **issues**. Cela a facilité la gestion des versions, la revue de code et le suivi des tâches à accomplir tout au long du projet.

The image is a screenshot of a GitHub repository page for a project named 'Chatbot Seatech'. The repository is owned by 'farah746' and is titled 'Update README.md'. The page shows the 'README.md' file with 34 lines of code. The main heading is 'Chatbot Seatech – Un assistant intelligent pour l'information académique'. Below this, there is a section titled 'Présentation du projet' which states the goal is to develop an intelligent chatbot for Seatech to help students and candidates access academic and administrative information. Another section titled 'Architecture et fonctionnement' describes the RAG (Retrieval-Augmented Generation) pipeline, listing steps such as data collection, chunking, embedding, storage, search, and response generation using various tools like Firecrawl, ChromaDB, and LLM Groq.

## **Conclusion**

Le développement de ce chatbot pour Seatech a permis d'améliorer considérablement l'accès aux informations pour les utilisateurs du site. Grâce à la méthode RAG et à l'utilisation du modèle Groc, nous avons pu garantir des réponses précises et adaptées aux besoins des utilisateurs. Ce projet représente une avancée significative dans la manière dont l'information est mise à disposition et consultée. Il constitue une première version du chatbot, et des améliorations futures sont déjà envisagées, notamment pour intégrer une meilleure gestion du contexte des conversations et optimiser davantage l'expérience utilisateur.